SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION **COURSE CODE: ITA0509**

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.* (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **Total Marks:50**

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I **N.AKSHAYA(192425129)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Q. No	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)	Total Score (100%)
1						
2						
3						
4						
5						
Total Marks (Out of 100)				Total Marks (Out of 50)		

Annexure A
Constraint-Based Problem Solving

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)

A company wants to deploy a **face recognition system** on low-power edge devices (e.g., Raspberry Pi). The system must:

A. Process live video streams in real time (<30 FPS latency)
B. Work under varying lighting conditions
C. Use limited memory and CPU resources
D. Detect and recognize faces from a predefined dataset

Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

Answer:

A. Process live video streams in real time (<30 FPS latency)

Use cv2.VideoCapture() to capture live video. Resize frames using cv2.resize() and process grayscale images to reduce computational load.

B. Work under varying lighting conditions

Use grayscale conversion followed by **CLAHE (Contrast Limited Adaptive Histogram Equalization)** to improve facial details under different lighting conditions.

C. Use limited memory and CPU resources

Use a lightweight **Haar Cascade Classifier** for face detection and process reduced-resolution frames instead of full-resolution images.

D. Detect and recognize faces from a predefined dataset

Detect faces using Haar Cascade, extract facial regions, and use **LBPH (Local Binary Patterns Histograms) Face Recognizer** trained on the predefined face dataset for recognition.

Complete Method:

Video Capture → Resize → Grayscale → CLAHE → Haar Cascade Face Detection → LBPH Face Recognition → Display Name

2. Automated Video Surveillance with Alert System

A security firm needs a **video surveillance system** that:

A. Detects motion and unusual activities
B. Generates alerts in real time
C. Works with standard CCTV video feeds
D. Minimizes false positives in dynamic environments

Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

Answer:

A. Detect motion and unusual activities

Use **Background Subtraction with MOG2** to identify moving objects from CCTV video. Use contours to determine the size and location of detected objects.

B. Generate alerts in real time

Define an activity threshold and generate an alert when a detected object remains within a suspicious region or crosses a predefined area.

C. Work with standard CCTV video feeds

Use cv2.VideoCapture() to read video from CCTV cameras or recorded CCTV files.

D. Minimize false positives in dynamic environments

Use **MOG2 background subtraction**, morphological opening/closing, and contour-area filtering to remove small noise and insignificant movements.

Complete Method:

CCTV Video → Frame Capture → MOG2 Background Subtraction → Morphological Filtering → Contour Detection → Activity Analysis → Alert

3. OCR System for Low-Quality Documents

A digitization company needs an **OCR system** that:

A. Extracts text from low-resolution, noisy scanned images
B. Handles varying fonts and illumination
C. Works efficiently for batch processing

Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

Answer:

A. Extract text from low-resolution, noisy scanned images

Use grayscale conversion, **Gaussian Blur/Median Blur**, and adaptive thresholding to remove noise and improve text visibility before OCR.

B. Handle varying fonts and illumination

Use **CLAHE** for contrast enhancement and **Adaptive Thresholding** to handle uneven illumination.

C. Work efficiently for batch processing

Process documents sequentially using an automated OpenCV preprocessing pipeline and send the processed images to an OCR engine.

D. Improve OCR recognition

Use morphological operations such as opening and closing to remove noise and connect broken characters before OCR.

Complete Method:

Input Document → Grayscale → Denoising → CLAHE → Adaptive Thresholding → Morphological Processing → Text Segmentation → OCR

4. Facial Expression Recognition for Mobile Application

A mobile app must detect **facial expressions (happy, sad, neutral)** with the following constraints:

A. Limited processing power (mobile CPU)
B. Real-time performance
C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

Answer:

A. Detect facial expressions: happy, sad, neutral

Use a **Haar Cascade Classifier** for face detection and a lightweight trained classifier/CNN for classifying the detected facial region into happy, sad, or neutral.

B. Handle limited processing power

Resize input frames and process only the detected face ROI. Use a lightweight face detector and expression classification model.

C. Achieve real-time performance

Process frames at a suitable reduced resolution and avoid unnecessary processing of the complete image.

D. Handle slight head movement and lighting changes

Use face ROI extraction, grayscale conversion, histogram/CLAHE enhancement, and normalization before expression classification.

Complete Method:

Camera → Resize → Face Detection → Face ROI → Grayscale → Lighting Enhancement → Normalization → Expression Classification → Output

5. Smart Traffic Monitoring System

A city authority needs a system to:

A. Detect vehicles from live video streams
B. Track movement across frames
C. Count vehicles and display statistics
D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Answer:

A. Detect vehicles from live video streams

Use **background subtraction or a lightweight object detector** to identify vehicles in each video frame.

B. Track movement across frames

Use **Centroid Tracking** or another object-tracking method to maintain the identity and position of vehicles across consecutive frames.

C. Count vehicles and display statistics

Create a **virtual counting line** using cv2.line(). Increment the vehicle count when a tracked vehicle crosses the line. Display the count using cv2.putText().

D. Operate continuously with minimal delay

Resize frames, process only required regions, and use efficient detection and tracking methods to maintain real-time performance.

Complete Method:

Live Video → Frame Preprocessing → Vehicle Detection → Centroid Tracking → Virtual Counting Line → Vehicle Count → Statistics Display

---

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)

Problem Understanding	**25%**	**Clearly defines the problem and identifies all relevant constraints accurately**	**Identifies most constraints with minor gaps**	**Partial understanding; misses key constraints**	**Fails to identify constraints correctly**
Constraint Analysis	**25%**	**Analyzes constraints effectively and prioritizes them logically**	**Provides reasonable analysis with some prioritization**	**Limited analysis; weak prioritization**	**No meaningful analysis or prioritization**
Solution Development	**25%**	**Develops optimal and feasible solutions satisfying all constraints**	**Develops workable solutions with minor limitations**	**Solutions partially satisfy constraints**	**Solutions do not meet constraints**
Application of Concepts	**15%**	**Effectively applies appropriate concepts, methods, or tools**	**Applies concepts with minor errors**	**Limited application; requires guidance**	**Unable to apply relevant concepts**
Justification	**10%**	**Provides strong justification for decisions with logical reasoning**	**Provides justification with some clarity**	**Weak or unclear justification**	**No justification provided**